



**FEU INSTITUTE OF TECHNOLOGY**

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

# **CS0065 (INTELLIGENT SYSTEMS)**

## **TECHNICAL ASSESSMENT **1****

### **RANDOM WALK SIMULATION USING AGENTPY**

|                     |                              |
|---------------------|------------------------------|
| <b>Student Name</b> | ELAZEGUI, LORENZO GABRIEL P. |
| <b>Section:</b>     | AN42                         |
| <b>Professor:</b>   | Ms. Crisola Tan              |
| <b>Date:</b>        | 10/02/2026                   |

## I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

## II. TECHNICAL ASSESSMENT

### RANDOM WALK SIMULATION USING AGENTPY

#### *Objective*

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

#### **Materials Needed**

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

#### **General Instructions**

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.

5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1\_lastname.pdf**  
(Example: TA1\_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

## Activity Instructions

### Step 1: Run the Code

1. Open the provided Python code.

#### Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# ----- Get user input from console -----
num_agents = int(input("Enter number of
agents: "))
grid_size = int(input("Enter grid size
(e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# ----- Agent Definition -----
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1,
        x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# ----- Model Definition -----
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
            random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()
```

```

# ----- Parameters from user input
----- parameters = {   'agents':
num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# ----- Run Model -----
model = RandomWalkModel(parameters)
model.setup()

# ----- Interactive Animation -----
--- fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1)) ax.grid(True)

scat = ax.scatter([], [], s=200) # s = size of agents

def update(frame):    model.step()    x =
[agent.position[0] for agent in
model.agents]    y = [agent.position[1]
for agent in model.agents]
    scat.set_offsets(list(zip(x,
y)))    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True,
repeat=False) plt.show()

```

2. Run the program in your terminal.
3. Enter the following values when prompted:
  - **Number of agents** (e.g., 5)
  - **Grid size** (e.g., 10 for a 10x10 grid) ○ **Number of steps** (e.g., 20)

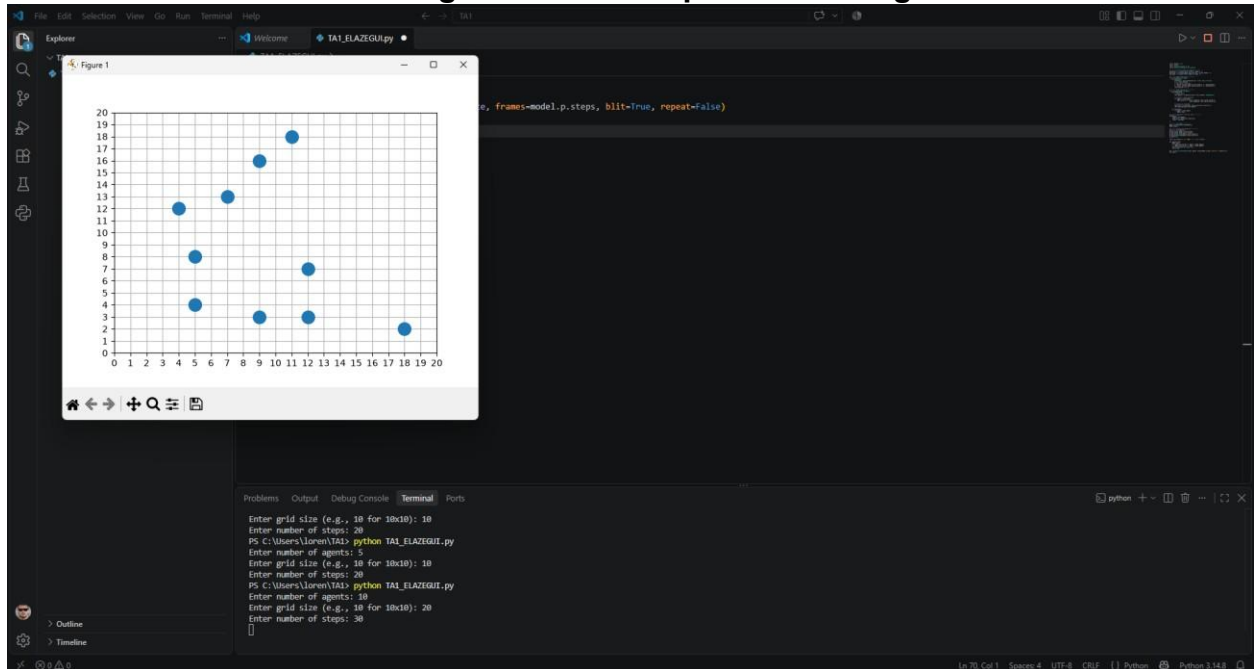
These values control the number of agents, the size of the environment, and the simulation duration.



freely across the grid, although some agents may occasionally cluster near the same area due to their random movement.

#### Step 4: Experiment with Parameters

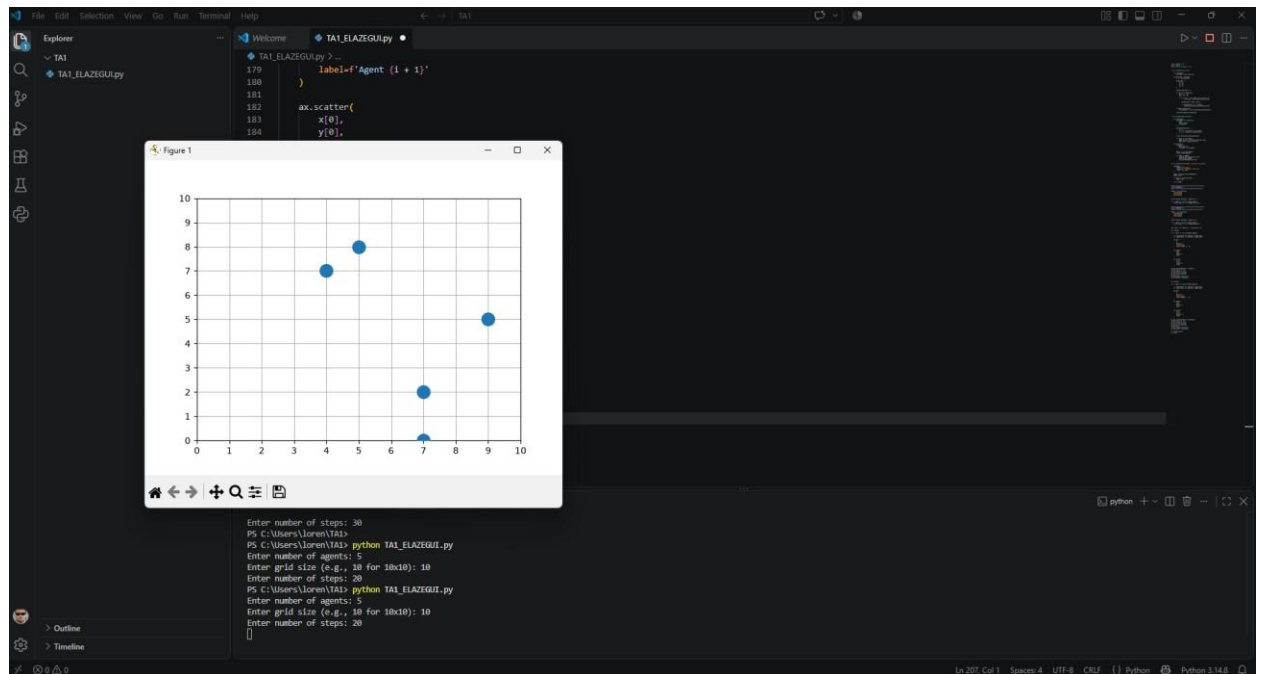
- Repeat the simulation with **different values**:
  - **More agents** (e.g., 10 or 20)
  - **Smaller/larger grid sizes**
  - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.



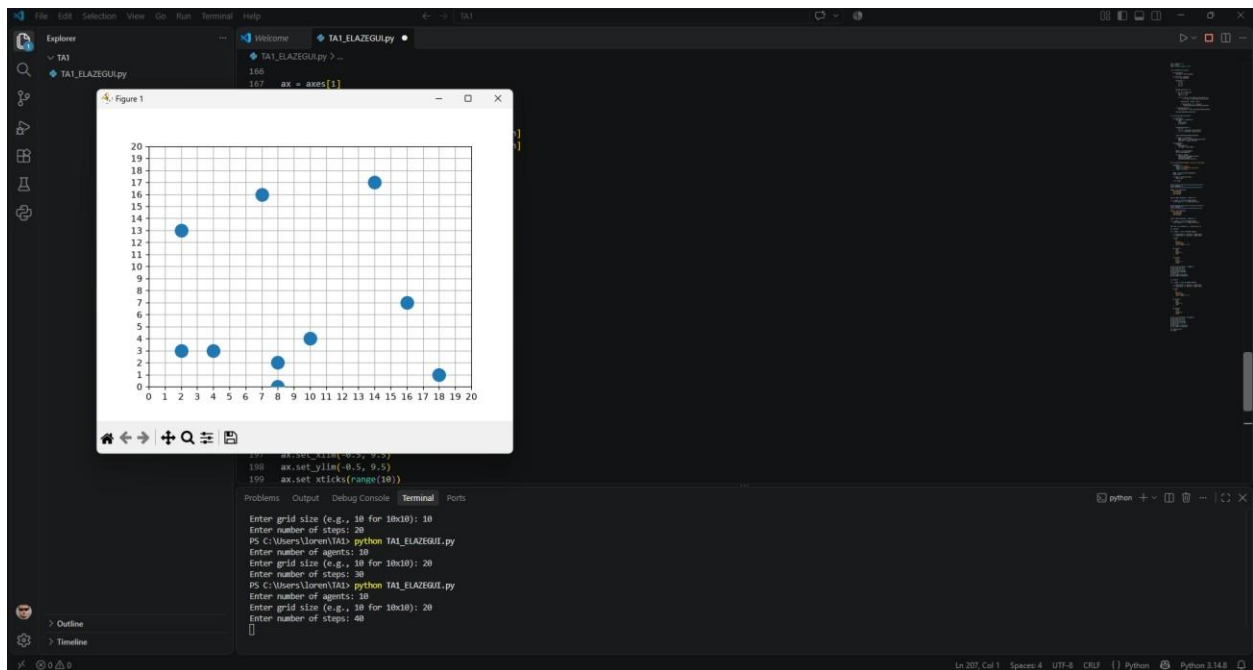
#### Laboratory Tasks

1. **Track agent paths:** Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules:** Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions:** Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios:** Run **two simulations with different parameters** and compare the movement patterns.

## Scenario 1



## Scenario 2



## Modified (Updated Code - Laboratory Tasks) ↓

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
```

```
class RandomWalker(ap.Agent):
```

```
    def setup(self):
        self.path = [self.position]
```

```
    def move(self, occupied):
        x, y = self.position
```

```
        directions = [
            (1, 0),
            (-1, 0),
            (0, 1),
            (0, -1)
        ]
```

```
        possible_positions = []
```

```
        for dx, dy in directions:
            new_x = x + dx
            new_y = y + dy
```

```

    if (0 <= new_x < self.model.p.grid_size[0] and
        0 <= new_y < self.model.p.grid_size[1]):

        new_position = (new_x, new_y)

        if new_position not in occupied:
            possible_positions.append(new_position)

    if possible_positions:
        self.position = random.choice(possible_positions)

    self.path.append(self.position)

```

```

class RandomWalkModel(ap.Model):

```

```

    def setup(self):
        self.agents = ap.AgentList(
            self,
            self.p.agents,
            RandomWalker
        )

        available_positions = [
            (x, y)
            for x in range(self.p.grid_size[0])
            for y in range(self.p.grid_size[1])
        ]

        random.shuffle(available_positions)

        for agent in self.agents:
            agent.position = available_positions.pop()
            agent.path = [agent.position]

    def step(self):
        occupied = {
            agent.position
            for agent in self.agents
        }

        agents = list(self.agents)
        random.shuffle(agents)

```

```

        for agent in agents:
            occupied.discard(agent.position)
            agent.move(occupied)
            occupied.add(agent.position)

def run_simulation(num_agents, grid_size, num_steps):

    parameters = {
        'agents': num_agents,
        'grid_size': (grid_size, grid_size),
        'steps': num_steps
    }

    model = RandomWalkModel(parameters)
    model.setup()

    for step in range(num_steps):
        model.step()

    return model

print("\n=====")
print("SCENARIO 1")
print("=====")

model1 = run_simulation(
    num_agents=5,
    grid_size=10,
    num_steps=20
)

print("\nFinal Positions - Scenario 1:")

for i, agent in enumerate(model1.agents):
    print(f"Agent {i + 1}: {agent.position}")

print("\n=====")
print("SCENARIO 2")
print("=====")

model2 = run_simulation(

```

```

    num_agents=10,
    grid_size=10,
    num_steps=20
)

print("\nFinal Positions - Scenario 2:")

for i, agent in enumerate(model2.agents):
    print(f"Agent {i + 1}: {agent.position}")

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

ax = axes[0]

for i, agent in enumerate(model1.agents):

    x = [position[0] for position in agent.path]
    y = [position[1] for position in agent.path]

    ax.plot(
        x,
        y,
        marker='o',
        markersize=3,
        label=f'Agent {i + 1}'
    )

    ax.scatter(
        x[0],
        y[0],
        marker='s',
        s=80
    )

    ax.scatter(
        x[-1],
        y[-1],
        marker='X',
        s=100
    )

ax.set_title("Scenario 1: 5 Agents")
ax.set_xlim(-0.5, 9.5)

```

```
ax.set_ylim(-0.5, 9.5)
ax.set_xticks(range(10))
ax.set_yticks(range(10))
ax.grid(True)
ax.set_xlabel("X Position")
ax.set_ylabel("Y Position")
```

```
ax = axes[1]
```

```
for i, agent in enumerate(model2.agents):
```

```
    x = [position[0] for position in agent.path]
    y = [position[1] for position in agent.path]
```

```
    ax.plot(
        x,
        y,
        marker='o',
        markersize=3,
        label=f'Agent {i + 1}'
    )
```

```
    ax.scatter(
        x[0],
        y[0],
        marker='s',
        s=80
    )
```

```
    ax.scatter(
        x[-1],
        y[-1],
        marker='X',
        s=100
    )
```

```
ax.set_title("Scenario 2: 10 Agents")
ax.set_xlim(-0.5, 9.5)
ax.set_ylim(-0.5, 9.5)
ax.set_xticks(range(10))
ax.set_yticks(range(10))
ax.grid(True)
ax.set_xlabel("X Position")
```

```
ax.set_ylabel("Y Position")
```

```
plt.tight_layout()
```

```
plt.show()
```

### Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

### Answer the Questions ↓

1. If the number of agents is very high compared to the grid size, the agents become more crowded and there are fewer available positions for movement. The agents may frequently encounter each other and have difficulty moving freely because they are programmed to avoid occupying the same position.
2. Increasing the number of steps gives the agents more opportunities to move around the grid. As a result, the agents can travel farther from their starting positions and explore more areas of the grid. However, their exact movement and final positions remain random.
3. No, the agents do not move outside the grid. The program checks whether a new position is within the grid boundaries before allowing an agent to move. Therefore, the agents are restricted to the available grid area.
4. This simulation can be used to represent situations where individuals or objects move randomly within a limited area. For example, it can model the movement of particles during diffusion, people moving through a crowded area, animals searching for food, or groups of individuals spreading through an environment. By changing the number of agents, grid size, movement rules, and number of steps, different real-life situations can be simulated.

